

Linux Homework Tasks

Task 1: Soft Link & Hard Link

- Learn the difference between soft links and hard links.
- Learn the commands to create both.
- Practice creating and deleting soft and hard links.
- Prepare for this as an interview question.

Task 2: adduser vs useradd

- Learn the difference between adduser and useradd.
- Understand which command is preferred on Ubuntu/Linux and why.
- Create a test user using the recommended command.

Task 3: journalctl

- Learn what journalctl is used for.
- Learn how to view system and service logs using journalctl.
- Practice checking logs for a specific service.

Task 4: Linux Command Cheat Sheet

- Review the Linux command cheat sheet.
- Practice the important commands covered in the cheat sheet.
- Understand the purpose and basic usage of each command.

Shell Scripting Homework Task

Task: System Information Script

Create a shell script that:

- Prints the current date.
- Prints the hostname.
- Prints the username.
- Prints the disk usage.
- Prints the running processes.
- Uses variables to store and use data.
- Takes user input using `read -p`.
- Creates a directory using `mkdir`.
- Creates a file using `touch`.
- Stores the running processes information in the file using `>` output redirection.

Commands to Use

- `mkdir`
- `touch`
- `echo`
- `df`
- `ps`
- `read -p`
- Variables
- `>` output redirection

Submission

- Create a public GitHub repository.
- Push the completed shell script to the repository.
- [Readme.md](#) file with all commands output.

Networking Homework Tasks

Task 1

- Practice commands and repo shared in devops-hero github repo.

Task 2

- Create an empty Markdown (`.md`) file.
- Execute the networking commands and add the output/screenshots to the file.
- Add a short explanation of what you understood about each command.

Git Homework Tasks

Task 1:

`git commit -a -m`

- Practice `git commit -a -m "message"`.
- Understand the difference between `git commit -a -m` and `git commit -m`.
- Test both commands and observe the difference.

Task 2: Git Cherry-Pick

- Create **2–4 commits** in the `main` branch.
- Use `git log` to view the commits.
- Create a new branch.
- Make **2–3 commits** in the new branch.
- Use `git log` to identify a specific commit.
- Cherry-pick one specific commit from the new branch into the `main` branch.
- Verify that the selected commit/change is now available in the `main` branch.

Submission

- Take screenshots of your work **or** create an `.md` file showing the commands and output.
- Upload the screenshots or `.md` file to the GitHub repository.

Docker Homework Tasks

Task: Hello World Applications

Create simple **Hello World** web applications using Docker for:

- Node.js application (preferably React)
- Python application
- Java application
- Apache web server
- React application
- Nginx application

Requirements

For each application:

- Create a separate folder:
 - `nodejs-app`
 - `python-app`
 - `java-app`
 - `Apache-app`
 - `React-app`
 - `nginx-app`
- Add the application code.
- Create a `Dockerfile`.
- Build the Docker image.
- Run the application using Docker.
- Verify that **Hello World** is displayed on a webpage.

Submission

- Push all applications and Dockerfiles to your GitHub repository.

- Maintain the folder structure mentioned above.

Docker Multi-Stage Build Homework

Task 1: Run Multi-Stage Dockerfile

- Clone the repository containing the multi-stage Dockerfile.
- Build the Docker image using the multi-stage Dockerfile.
- Run a container from the image.
- Access the application running inside the container.
- Verify that the application displays:
Hello World from Docker multi-stage build
- Verify the running container using `docker ps`.
- Confirm that the application is running on port `8080`.

Task 2: Documentation

Create an `.md` file containing:

- Your name
- Your enrollment number
- Screenshot or output showing the application running successfully.
- Screenshot or output of `docker ps` showing the running container on port `8080`.

Task 3: Docker Application Deployment

Deploy at least **3 different types of applications** using Docker, such as:

- Node.js
- Python
- Java

Submission

- Upload the completed Markdown file to your GitHub repository.
- Include the required screenshots or command outputs as evidence.

Docker Networking & Volume Homework Tasks

Task 1: Docker Container Networking

- Create 3 containers:
 - Frontend
 - Backend
 - Database
- Use Nginx or Alpine images for the frontend and backend.
- Use the MySQL image for the database.
- Create 3 different Docker networks.
- Add the backend container to 2 networks.
- Check connectivity between the containers.

Task 2: Host Network

- Pull the Apache2 image from Docker Hub.
- Create an Apache2 container using the host network.
- Access the Apache website directly on port 80.

Task 3: Bind Mount

- Create a folder on your local machine.
- Create an `index.html` file with **Hello students** as the content.
- Bind mount the folder to an Nginx container.
- Access the Nginx website and verify the content.
- Modify the `index.html` file.
- Verify that the changes are reflected without restarting the container.

Task 4: Overlay Network

- Research Docker overlay networks.
- Understand their use cases.

- Understand how overlay networks work across multiple Docker hosts.

Submission

- Take screenshots of the completed exercises.
- Add the screenshots to the [README.md](#) file in your GitHub repository.
- Complete any remaining exercises from the session.

Kubernetes Fundamentals: Resources

- [Kubernetes Basics Tutorial](#)
- [Minikube Installation Guide](#)
- [Kubernetes Architecture](#)
- [Kubernetes GitHub Repository](#)

Task

1. Install and configure Minikube.
2. Verify Kubernetes cluster status.
3. Explore Kubernetes architecture.
4. Learn the basic Kubernetes objects and commands.
5. Perform the Kubernetes Basics tutorial hands-on.

Deliverables

- Commands used
- Output screenshots
- Short notes on Kubernetes architecture

- Push your work to GitHub

Session 10: Kubernetes Pods, ReplicaSets & Deployments

Task 1: Deployment Strategies

Implement all **4 deployment strategies**:

01. Rolling Update

- Create Deployment
- Configure rolling update
- Perform an application update
- Verify old and new Pods

02. Blue-Green Deployment

- Create Blue version
- Create Green version
- Switch traffic between versions
- Verify the active version

03. Canary Deployment

- Deploy stable version
- Deploy canary version
- Route a small percentage of traffic to Canary
- Verify both versions

04. Recreate Deployment

- Deploy the application
- Update the application
- Observe the old Pods being terminated before new Pods are created

Task 2: Pod Lifecycle

Study and demonstrate the Kubernetes Pod lifecycle.

For **each** YAML file:

1. Apply the YAML file.
2. Check Pod status.
3. Check Pod details.
4. Capture the output.
5. Add the screenshot to README.md.
6. Explain what you observed.

Deliverables

- YAML files
- Commands
- Command outputs
- Screenshots
- Explanation in README.md

Session 11: Kubernetes Networking & Services

Task 1: Kubernetes Services

Deploy and demonstrate all **5 Service types**:

1. ClusterIP
2. NodePort
3. LoadBalancer
4. ExternalName
5. Headless

For each Service:

- Create the required YAML
- Deploy the application
- Verify the Service
- Test connectivity
- Capture output
- Add output/screenshots to README.md

Task 2: Kubernetes Object Comparison

Create a README.md explaining:

Deployment vs ReplicaSet

Explain:

- Purpose
- Pod management
- Scaling
- Rolling updates
- Relationship between Deployment and ReplicaSet

Deployment vs DaemonSet vs StatefulSet

Compare:

- Use cases
- Pod creation
- Scaling
- Networking
- Storage

- Examples

ReplicaSet vs Service

Explain:

- ReplicaSet responsibility
- Service responsibility
- Why a Service is required
- How traffic reaches Pods

Task 3: FQDN

Create:

fqdn/

└─ README.md

Document:

- What is FQDN?
- Kubernetes Service DNS
- Kubernetes DNS naming convention
- Namespace-based DNS
- Pod-to-Service communication
- Examples of Kubernetes FQDNs

Task 4: CoreDNS

Research and document:

- What is CoreDNS?
- Why Kubernetes uses CoreDNS
- How Service discovery works
- How DNS queries are resolved
- CoreDNS configuration
- How to troubleshoot DNS issues

Deliverables

- Service YAML files
- Comparison documentation
- fqdn/README.md
- coredns/README.md
- Screenshots/output

Session 12: Kubernetes Ingress, ConfigMaps & Secrets

Task 1: ConfigMap

Perform a complete hands-on demo:

- Create ConfigMap
- Store configuration values
- Inject ConfigMap into Pod
- Verify values inside the container

Task 2: Secret

Perform a complete hands-on demo:

- Create Secret
- Store sensitive values
- Inject Secret into Pod
- Verify the value inside the container
- Understand why Secrets should not be committed directly to Git

Task 3: Ingress

Perform a complete Ingress demo:

- Deploy application
- Create Service
- Configure Ingress
- Access application through Ingress
- Verify routing

Task 4: Ingress vs Ingress Controller

Create a README.md explaining:

- What is Ingress?
- What is an Ingress Controller?
- Difference between them
- Why both are required
- Examples

Task 5: Troubleshooting

Use the troubleshooting folder and:

1. Identify the problem.
2. Run troubleshooting commands.
3. Find the root cause.
4. Fix the issue.
5. Capture before/after output.

6. Add screenshots to README.md.

Deliverables

- ConfigMap YAML
- Secret YAML
- Ingress YAML
- Troubleshooting documentation
- Screenshots
- README.md

Session 13: Kubernetes Storage, HPA & Probes

Task 1: Kubernetes Volumes

Create:

01-kubernetes-volumes/

└─ README.md

Document what you learned about:

- emptyDir
- hostPath
- PersistentVolume
- PersistentVolumeClaim
- StorageClass
- Dynamic provisioning

Include practical examples wherever possible.

Task 2: HPA Hands-on

Use hpa.yml and perform the following:

1. Deploy the application.
2. Configure HPA.
3. Verify HPA.
4. Deploy a load generator.
5. Increase application load.
6. Observe CPU utilization.
7. Observe Pod scaling.
8. Capture the output.
9. Add the output/screenshots to README.md.

Useful commands should include:

```
kubectl get hpa  
kubectl get pods  
kubectl top pods  
kubectl describe hpa
```

Task 3: Mini Project

Complete the mini project provided for Session 13.

Deliverables

- Volume documentation
- HPA YAML
- Load generator
- HPA output
- Screenshots
- Mini-project implementation
- README documentation

Session 14: Kubernetes Troubleshooting

Task 1: Kubernetes Commands

Perform hands-on practice with all important troubleshooting commands.

Cover:

kubectl get

kubectl describe

kubectl logs

kubectl exec

kubectl events

kubectl explain

kubectl top

kubectl get -o wide

Task 2: Troubleshoot Common Issues

Practice troubleshooting:

- CrashLoopBackOff
- ImagePullBackOff

- ErrImagePull
- Pending
- ContainerCreating
- Service connectivity issues
- DNS issues
- Pod networking issues
- Configuration issues

For every issue:

1. Identify the problem.
2. Investigate.
3. Find the root cause.
4. Fix it.
5. Verify the solution.
6. Document the troubleshooting process.

Task 3: Mini Project

Complete the Kubernetes troubleshooting mini project.

Deliverables

- Commands
- Problem statement
- Investigation steps
- Root cause
- Solution
- Before/after output
- Screenshots
- README.md

Session 15: Helm

Task 1: Helm Commands

Perform hands-on practice with all important Helm commands covered in the session.

For every command:

- Execute the command.
- Understand what it does.
- Capture the output.
- Document it in the relevant README.md.

Cover:

helm create

helm install

helm list

helm status

helm get

helm upgrade

helm history

helm rollback

helm uninstall

helm repo

helm search

Task 2: Helm Rollback

Perform a complete rollback workflow:

Install



Upgrade



Verify



Upgrade again



Verify



Rollback



Verify

Document the complete process.

Task 3: Mini Project

Complete the Helm mini project.

Deliverables

- Helm chart
- values.yaml
- Templates
- Installation
- Upgrade
- Rollback
- Screenshots
- README files
- Mini project

Session 16: CI/CD & GitHub Actions

Task: Demo Project

Build a complete CI/CD demo project using GitHub Actions.

Refer to:

10-final-cicd-pipeline

The project should cover:

- CI vs CD
- CI/CD pipeline
- GitHub Actions
- Workflow
- Jobs
- Steps
- Runners
- Secrets
- Artifacts
- Build
- Test

- Pipeline execution

Deliverables

- Application source code
- Dockerfile
- GitHub Actions workflow
- CI pipeline
- CD pipeline
- Screenshots of successful pipeline execution
- README.md

Session 17: Complete CI/CD & DevSecOps

Task: DevSecOps Demo Project

Build a complete CI/CD + DevSecOps pipeline.

The project should include:

CI/CD

- Application build
- Unit testing
- Docker image build
- Container registry
- Kubernetes deployment

Security

- SAST
- SCA
- Secret scanning
- Container image scanning

- Security gates

Expected Flow

Code



Build



Unit Test



SAST



SCA



Secret Scan



Docker Build



Container Image Scan



Security Gate



Push Image



Deploy to Kubernetes

Deliverables

- Application
- Dockerfile
- GitHub Actions workflow
- Security tools configuration
- Kubernetes manifests
- Successful pipeline output
- Screenshots
- Complete README.md

Session 18: Terraform & Infrastructure as Code

Task 1: Terraform S3 Demo

Create a Terraform project:

terraform-s3-demo/

- |— main.tf
- |— variables.tf
- |— outputs.tf
- |— provider.tf
- |— terraform.tfvars
- |— [README.md](#)

Create an **AWS S3 bucket** using Terraform.

Perform:

terraform init

terraform fmt

terraform validate

terraform plan

terraform apply

terraform show

terraform output

terraform destroy

Document the complete workflow in [README.md](#).

Task 2: AWS Services Research

Learn about the following AWS services and create a separate README.md for each.

01. IAM - Governance

Learn and document:

- What is IAM?
- Users
- Groups
- Roles

- Policies
- Permissions
- Least privilege
- IAM best practices
- Common use cases

02. EC2 - Compute

Learn and document:

- What is EC2?
- AMI
- Instance types
- Key pairs
- Security Groups
- EBS
- Public vs private IP
- Instance lifecycle
- Common use cases

03. S3 - Storage

Learn and document:

- What is S3?

- Buckets
- Objects
- Storage classes
- Versioning
- Lifecycle policies
- Encryption
- Bucket policies
- Common use cases

04. VPC - Networking

Learn and document:

- What is VPC?
- CIDR
- Subnets
- Route tables
- Internet Gateway
- NAT Gateway
- Security Groups
- Network ACLs
- Public vs private subnet

05. DynamoDB & RDS - Database Services

Learn and document:

DynamoDB

- NoSQL
- Tables
- Items
- Attributes
- Partition key
- Sort key
- Use cases

RDS

- Relational database
- Supported engines
- DB instances
- Security
- Backups
- Multi-AZ
- Read replicas

- Use cases

Deliverables

terraform-s3-demo/

aws-services/

|— 01-iam/

| |— README.md

|— 02-ec2/

| |— README.md

|— 03-s3/

| |— README.md

|— 04-vpc/

| |— README.md

|— 05-dynamodb-rds/

 |— README.md

Session 19: Cloud & Terraform in Action

Task

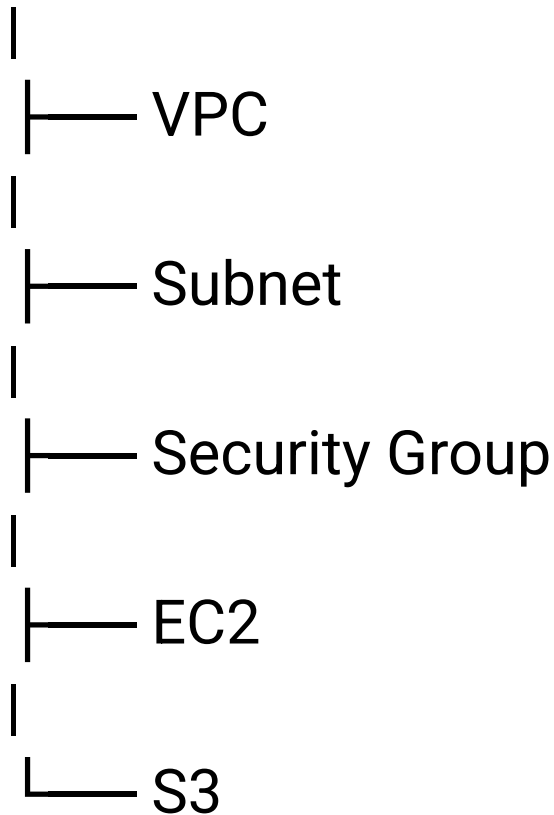
Build an end-to-end cloud infrastructure project using Terraform.

The project should demonstrate:

- Terraform providers
- Variables
- Resources
- Outputs
- Dependencies
- AWS infrastructure
- Terraform state
- terraform plan
- terraform apply
- terraform destroy

Suggested Architecture

Terraform



Deliverables

- Terraform project
- AWS resources
- Architecture diagram
- Screenshots
- Terraform commands
- README.md

Session 20: Monitoring, Observability & GitOps

Task 1: Monitoring

Learn and demonstrate:

- Metrics
- Logs
- Alerts
- CPU utilization
- Memory utilization
- Application health

Task 2: Observability

Understand the three major pillars:

Metrics

Logs

Traces

Document:

- What each pillar means
- Why observability is required

- Common tools
- Kubernetes observability

Task 3: GitOps

Learn:

- What is GitOps?
- Git as the source of truth
- Declarative configuration
- Continuous reconciliation
- GitOps workflow
- Kubernetes + GitOps

Deliverables

- Monitoring demo
- Observability documentation
- GitOps demo
- Screenshots
- README.md

Session 21: Final DevOps Project & Troubleshooting

Final Project

Build a complete **end-to-end DevOps project** using the concepts learned throughout the course.

Project should include

Application



Git



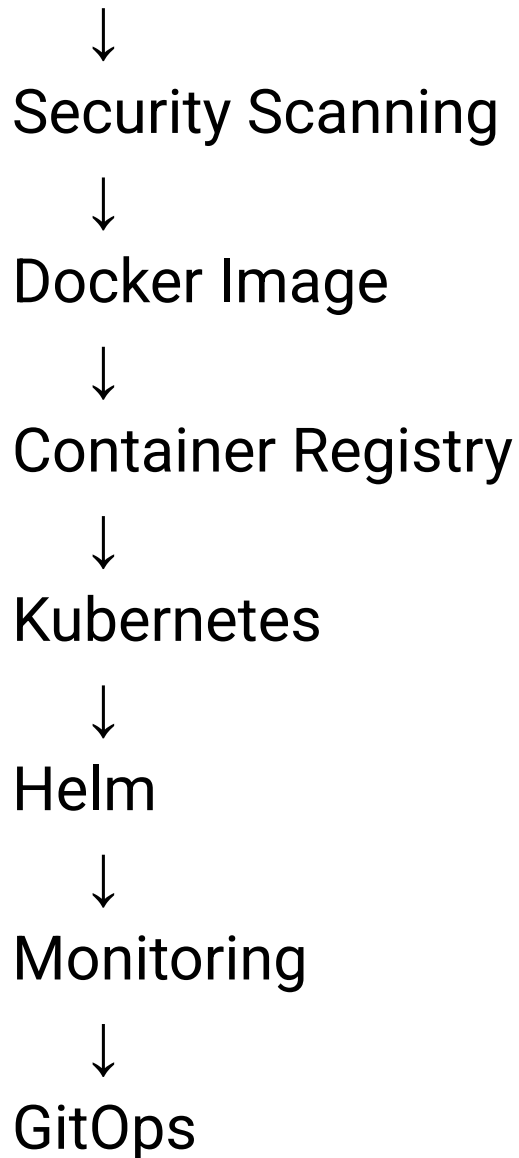
GitHub



CI Pipeline



Build & Test



Infrastructure

Use Terraform to provision the required cloud infrastructure.

Kubernetes

Include:

- Deployment

- Service
- ConfigMap
- Secret
- Ingress
- HPA
- Probes
- Storage where required

CI/CD

Include:

- GitHub Actions
- Build
- Test
- Docker build
- Image push
- Kubernetes deployment

DevSecOps

Include:

- SAST
- SCA

- Secret scanning
- Container image scanning
- Security gates

Monitoring & GitOps

Include:

- Monitoring
- Logs/metrics
- GitOps workflow

Final Troubleshooting Challenge

Intentionally introduce multiple issues into the project.

Students must:

1. Identify the issue.
2. Investigate the logs and resources.
3. Find the root cause.
4. Fix the issue.
5. Verify the solution.
6. Document the troubleshooting process.

Final Deliverables

final-devops-project/

```
|— application/  
|— docker/  
|— kubernetes/  
|— helm/  
|— terraform/  
|— .github/  
|   |— workflows/  
|— security/  
|— monitoring/  
|— gitops/  
|— README.md
```

The final README.md should contain:

- Project overview
- Architecture diagram
- Technologies used
- Application setup
- Docker setup
- Kubernetes deployment

- Helm deployment
- Terraform infrastructure
- CI/CD pipeline
- DevSecOps implementation
- Monitoring
- GitOps
- Troubleshooting
- Screenshots
- Lessons learned